משימה 1 - השידור המיורט

נושאים: מחרוזות, מערכים, פונקציות, מצביעים בסיסיים וקלט.

הרצת `./operation_b7/` מהתיקייה הראשית יוצרת את `mission1/data/transmissions.txt`. אין לערוך את קובץ הראיות שנוצר.

תדריך מודיעיני

השידור שבו ד"ר דופנשמירץ הכריז על הבארשבעינייטור נקטע לפני שהסביר מה הוא מתכנן לעשות לבאר שבע - כנראה מפני שגם לנבלים יש בעיות קליטה. כוחות הביטחון הצליחו ליירט שיחות ממערכת התקשורת של הארגון, אך ההודעות מוצפנות. פענוחן הוא הדרך היחידה להבין מי מעורב, מה מתוכנן ולאן תוביל החקירה. למרבה המזל, התקבל גם קטע משיחה פנימית:

> **פלנקטון:** סיימתי את מערכת ההצפנה החדשה! אף אחד לא יצליח לפצח אותה!
>
> **מורטי הרשע:** באיזה אלגוריתם השתמשת?
>
> **פלנקטון:** שתי לולאות `for`!
>
> **מורטי הרשע:** ...
>
> **פלנקטון:** קודם מזיזים את האותיות באלפבית, ואז הופכים את המחרוזת!
>
> **מורטי הרשע:** ואת המפתח?
>
> **פלנקטון:** שמרתי ליד כל הודעה, כדי שלא נאבד אותו.

התשדורות המיורטות הן כרגע קצה החוט היחיד שלנו. עליכם לפענח אותן, להפריד בין הודעות תקינות למידע פגום ולגלות מי עומד מאחורי המתקפה על באר שבע. כל מידע שתצליחו לשחזר עשוי להיות קריטי להמשך החקירה.

חלק א׳ - פענוח הודעה

פלנקטון מצפין הודעה בשני שלבים:

1. מזיז כל אות אנגלית גדולה קדימה לפי מפתח 0–25, עם מעבר מחזורי מ־`Z` ל־`A`. התו `_` אינו מוזז.
2. הופך את כל המחרוזת, כולל התווים `_`.

לדוגמה:

```
Original:     HELLO_C
Shift:        3
After shift:  KHOOR_F
Encrypted:    F_ROOHK
```

ממשו ב־`decoder.c`:

```
int decode_message(const char *encoded, int shift, char *decoded, size_t
capacity);
```

- כתבו את ההודעה המפוענחת ל־`decoded` בלי לשנות את `encoded`.
- הפרמטר `capacity` הוא קיבולת מערך הפלט, כולל מקום ל־`'\0'`.
- החזירו `1` בהצלחה ו־`0` אם אחד המצביעים הוא `NULL`, המפתח אינו בטווח, מופיע תו שאינו `A-Z` או `_`, או אין מקום לפלט.
- מחרוזת ריקה חוקית, בתנאי שקיים מקום לתו הסיום.
- ניתן להניח שאין חפיפה בזיכרון בין המקור ליעד. אין לכתוב מחוץ לגבולות מערך הפלט.

לבדיקה, מתוך התיקייה `/mission1`:

```
make test-a
```

המפענח הראשוני מוכן, אבל תיק החקירה מכיל גם תשדורות שאולי אינן ראיות אמינות. כדי להפריד בין מידע אמיתי להסחות דעת, עליכם לבדוק כעת את הרשומה המלאה.

חלק ב׳ - ניתוח תשדורת ואימותה

כל תשדורת מופיעה בשורה אחת, בפורמט:

```
SENDER|KEY|CHECKSUM|ENCRYPTED_MESSAGE
```

Field	Requirements
`SENDER`	1–24 characters; uppercase letters (`A-Z`) and underscores (`_`) only
`KEY`	Decimal digits representing a value between 0 and 25
`CHECKSUM`	Exactly 2 decimal digits; value between 00 and 96
`ENCRYPTED_MESSAGE`	1–160 characters; uppercase letters (`A-Z`) and underscores (`_`) only

ברשומה תקינה יש **בדיוק שלושה** מפרידי `|` וללא תווי רווח.

ה־Checksum מחושב על ההודעה **לאחר הפענוח**: סכום ערכי ה־ASCII של כל התווים, מודולו 97. אין לכלול בחישוב את `\0`.

ממשו:

```
int calculate_checksum(const char *message);
int process_transmission(const char *line);
```

- הפונקציה `calculate_checksum` תחזיר את סכום הביקורת, או `-1` אם `message == NULL`.

- הפונקציה `process_transmission` מקבלת שורה אחת ללא תווי סוף־שורה ואינה משנה אותה. עליה לבדוק את מבנה הרשומה, לפענח את ההודעה ולאמת את ה־Checksum.
- לרשומה תקינה, הדפיסו `[SENDER]` ואז את `DECODED_MESSAGE` והחזירו `1`.
- לרשומה לא תקינה, הדפיסו `[INVALID]` והחזירו `0`.
- אין להשתמש ב־`sscanf` לפיצול הרשומה.

רמז: ניתן להיעזר ב־`strtok` לצורך פיצול הרשומה לשדות. זכרו שהפונקציה משנה את המחרוזת שהיא מקבלת, ולכן יש להעתיק תחילה את הקלט למערך מקומי בגודל מתאים.

לבדיקה:

```
make test-b
```

כעת אתם יודעים לפענח ולאמת תשדורת אחת. הגיע הזמן להפעיל את הכלי ברצף, לעבור על כל תיק הראיות ולשחזר את השיחה המיורטת.

חלק ג׳ - שחזור השיחה

ממשו את `main` כך שתקרא שורות מ־`stdin` באמצעות `fgets` עד `EOF`. לפני עיבוד כל שורה, הסירו את תווי סוף השורה, אם קיימים (`\r\n` או `\n`). העבירו כל רשומה לפונקציה `process_transmission` וספרו כמה רשומות התקבלו וכמה נדחו.

ניתן להניח שאורך כל שורה אינו עולה על 200 תווים, לא כולל תווי סוף־שורה. יש לטפל גם בשורה האחרונה אם אינה מסתיימת ב־`newline`.

בסיום הדפיסו בדיוק:

```
Recovered: N | Rejected: M
```

דוגמה:

קלט:

```
TEST|3|49|F_ROOHK
TEST|3|48|F_ROOHK
```

פלט:

```
[TEST] HELLO_C
[INVALID]
Recovered: 1 | Rejected: 1
```

לבדיקה:

```
make test-c
```

לאחר מכן, מתוך `/mission1`, קמפלו והריצו על הראיות שלכם:

```
make run
```

אין להוסיף הודעות דיבוג או קישוטים לפלט התוכנית. אם הבדיקות נכשלות, תקנו את המימוש לפני הפעלתו על התשדורות המיורטות.

חילוץ קוד הגישה

בין ההודעות **התקינות** מופיעים שני חלקים של קוד גישה: `PART_ONE` ו־`PART_TWO`. חברו אותם לפי הסדר וכתבו בדוח את הקוד בפורמט:

```
B7-M1-XXXXXXXX
```

שימו לב: לא כל תשדורת היא ראיה מהימנה. רשומה שנכשלה בבדיקת ה־Checksum אינה יכולה לשמש לחילוץ הקוד.

קוד הגישה סוגר את שלב היירוט, אבל לא את התיק: השמות והפעולות שתמצאו בתשדורות הם נקודת הפתיחה לחקירת מאגר הסוכנים ויומן הגישה במשימה הבאה.

דוח החקירה

מלאו את החלק של משימה 1 ב־`investigation_report.md`.

ממצאים: מה גיליתם בתשדורות, ומהו הממצא החשוב ביותר להמשך החקירה? ציינו את קוד הגישה שחילצתם.

הבנה טכנית: הביאו דוגמה אחת לרשומה פגומה (מהראיות או דוגמה שיצרתם), והסבירו מה גורם ל־`process_transmission` לדחות אותה.

אין צורך לתאר את אלגוריתם הפענוח כולו או להסביר את הקוד שורה אחר שורה. תשובות קצרות, מדויקות ומנומקות מספיקות.

אתגר רשות - פלנקטון משדרג את ההצפנה

בגרסה זו משתמשים במערך מפתחות מחזורי. התו שבמיקום `i` בהודעה המקורית מוזז לפי `keys[i % key_count]`; נספר כמיקום אך אינו מוזז. בסיום הופכים את המחרוזת.

ממשו:

```
int decode_advanced(const char *encoded, const int *keys, size_t key_count, char
*decoded, size_t capacity);
```

כל המפתחות חייבים להיות בתחום 0–25, `key_count` חייב להיות חיובי, ונדרשות אותן בדיקות מצביעים, תווים וקיבולת כמו בחלק א׳. לדוגמה, `HELLO_C` עם המפתחות `[1, 2, 3]` מוצפן ל־`D_QMOGI`.

האתגר אינו חלק מחובת ההגשה.

הגשה

הגישו את:

- `mission1/decoder.c`
- `mission1/main.c`
- `investigation_report.md` (עם החלק של משימה 1)

אין לשנות חתימות פונקציות ב־`decoder.h` או להגיש קובצי הרצה.